Ejercicios Tema 5. Diseño y realización de pruebas con JUnit

Objetivos

1. Analizar código ya implementado.
2. Calcular la complejidad ciclomática de métodos sencillos.
3. Diseñar casos de prueba que cubran todas las ramas.
4. Implementar pruebas unitarias con JUnit.
5. Diseñar pequeños programas propios y justificar sus pruebas a partir de la complejidad del código.

Consideraciones

En **todos los ejercicios** debes entregar:

1. **Análisis del método o programa**
   - Qué hace.
   - Qué entradas admite.
   - Qué salidas produce.
   - Casos normales y casos límite.
2. **Complejidad ciclomática**
   - Cálculo razonado del número mínimo de tests para cubrir las decisiones.
3. **Diseño de casos de prueba**

   Para cada caso:
   - ID
   - Descripción
   - Entrada
   - Resultado esperado
   - Oráculo
   - Rama o camino que cubre
4. **Implementación en JUnit**
   - Clase `...Test` en `src/test/java`
   - Métodos de test bien nombrados

- Uso de `assertEquals`, `assertTrue`, `assertThrows`, `assertDoesNotThrow` cuando proceda.

Plantilla para los casos de prueba en cada ejercicio

ID	Descripción	Entrada	Esperado	Estado
TC01				
TC02				
TC03				
...				

Ejercicio 1. Método signo

Se proporciona el siguiente método

```
public static int signo(int x) {
    if (x > 0) {
        return 1;
    } else if (x < 0) {
        return -1;
    }
    return 0;
}
```

Tareas

1. Calcula la complejidad ciclomática.
2. Diseña el conjunto mínimo de pruebas para cubrir todas las ramas.
3. Implementa la clase `SignoTest` con JUnit.

Ejercicio 2. Método clasificarEdad

Se proporciona el siguiente método:

```
public static String clasificarEdad(int edad) {
    if (edad < 0) {
        throw new IllegalArgumentException("Edad no válida");
    }

    if (edad < 6) {
        return "Infancia";
    } else if (edad < 12) {
        return "Niñez";
    } else if (edad < 18) {
        return "Adolescencia";
```

```
    } else if (edad < 25) {
        return "Juventud";
    } else if (edad < 60) {
        return "Adultez";
    } else {
        return "Vejez";
    }
}
```

Tareas

1. Calcula la complejidad ciclomática.
2. Diseña el conjunto mínimo de pruebas para cubrir todas las ramas.
3. Añade además casos límite especialmente relevantes.
4. Implementa `ClasificarEdadTest`.

Ejercicio 3. Método contarPositivos

Se proporciona el siguiente método:

```
public static int contarPositivos(int[] datos) {
    int contador = 0;
    for (int i = 0; i < datos.length; i++) {
        if (datos[i] > 0) {
            contador++;
        }
    }
    return contador;
}
```

Tareas

1. Calcula la complejidad ciclomática.
2. Diseña pruebas para cubrir:
    - array vacío,
    - array con todos negativos,
    - array con mezcla de positivos, negativos y ceros.
3. Justifica qué ramas cubre cada prueba.
4. Implementa `ContarPositivosTest`.

Ejercicio 4. Método calificacion

Se proporciona el siguiente método:

```
public static String calificacion(int nota) {
    if (nota < 0 || nota > 10) {
        throw new IllegalArgumentException("Nota no válida");
    }

    return switch (nota) {
        case 0, 1, 2, 3, 4 -> "SUSPENSO";
        case 5 -> "SUFICIENTE";
        case 6 -> "BIEN";
        case 7, 8 -> "NOTABLE";
        default -> "SOBRESALIENTE";
    };
}
```

Tareas

1. Calcula la complejidad ciclomática
2. Diseña los casos mínimos para cubrir todas las alternativas del `switch` y la validación inicial.
3. Implementa `CalificacionTest`.
4. Indica qué casos podrían agruparse con `@ParameterizedTest`.

Ejercicio 5. Crear un método esBisiesto

Implementa un método con la siguiente firma:

```
public static boolean esBisiesto(int anyo)
```

Reglas

Un año es bisiesto si:

- es divisible por 4 y no por 100, o
- es divisible por 400.

Tareas

1. Implementa el método.
2. Calcula su complejidad ciclomática a partir de tu código.
3. Diseña los casos de prueba necesarios para cubrir todas las decisiones.
4. Implementa `EsBisiestoTest`.

Requisitos

Debes incluir al menos estos tipos de caso:

- divisible por 4 y no por 100,
- divisible por 100 y no por 400,
- divisible por 400,

- no divisible por 4.

Ejercicio 6. Implementar Piedra – Papel – Tijera

Implementa el método:

```
public static String jugar(String jugador1, String jugador2)
```

Las jugadas posibles son:

- `"PIEDRA"`
- `"PAPEL"`
- `"TIJERA"`

Reglas

- Piedra gana a tijera
- Tijera gana a papel
- Papel gana a piedra
- Si ambas jugadas son iguales --> `"EMPATE"`
- Si alguna jugada no es válida --> `IllegalArgumentException`

Tareas

1. Implementar el método.
2. Calcular la complejidad ciclomática.
3. Diseñar los tests mínimos para cubrir todas las ramas.
4. Implementar `PiedraPapelTijeraTest`.

Ejercicio 7. Implementar Validador de contraseñas

Implementa el método:

```
public static int evaluarPassword(String password)
```

Reglas

Se asignan puntos:

- +1 si tiene al menos **8 caracteres**
- +1 si tiene **mayúsculas**
- +1 si tiene **minúsculas**
- +1 si tiene **números**
- +1 si tiene **símbolos**

Interpretación del resultado:

- entre 0 y 2 --> "DEBIL"
- entre 3 y 4 --> "MEDIA"

- 5 --> "FUERTE"

Tareas

1. Implementar el método.
2. Calcular complejidad ciclomática.
3. Diseñar casos de prueba representativos.
4. Implementar `PasswordTest`.